MALATYA TURGUT ÖZAL ÜNİVERSİTESİ
Bilgisayar Mühendisliği Bölümü

MOBİL PROGRAMLAMA
Dönem Projesi Raporu



NAYA

S1000D teknik dokümanlarını 3D modellerle birleştiren, yapay zekâ destekli arıza teşhis ve bakım uygulaması

HAZIRLAYAN
Enes Yeniçırak
02230202051

HAZIRLAYAN
Muhammet Mustafa Zahter
02230202060

TARİH · SÜRÜM
Ekim 2026
v1.0 · 01 / 14

İçindekiler

01 **Giriş** — Problem, amaç ve kapsam — 03

02 **Genel Tanıtım** — İki yön ve bir kullanım senaryosu — 04

03 **S1000D ve .naya Formatı** — Standart, data module kodu ve kendi dosya dilimiz — 05

04 **Sistem Mimarisi** — Uygulama, sunucu, veri ve yapay zekâ katmanları — 06

05 **Flutter Katmanı** — Katmanlı yapı, paketler ve platform farkları — 07

06 **Veritabanı** — Tablolar, ilişkiler ve güvenlik — 08

07 **Çalışma Mantığı ve Algoritmalar** — Teşhis akışı ve içe aktarma — 09

08 **Arayüz ve Kullanıcı Akışı** — Ekranlar, model sayfası, açılış — 10

09 **Dosya Yapısı** — Monorepo ve uygulama klasörleri — 11

10 **Çalışma Şekli** — Notion, sıra döngüsü, dal ve PR kuralları — 12

11 **Görev Dağılımı ve Takvim** — Tarihli adımlar ve sorumlular — 13

12 **Sonuç ve Kaynaklar** — Beklenen çıktılar ve başvurulan kaynaklar — 14

Raporun her sayfası bir teknik çizim paftası gibi düzenlendi. Üstteki kod, S1000D'deki **data module kodunun** biçimini taklit eder; sağ alttaki antet bloğu pafta numarasını gösterir.

01 Giriş

Bir aracın, bisikletin ya da uçağın bakım bilgisi binlerce sayfalık teknik dokümanların içinde durur. NAYA bu bilgiyi aracın üç boyutlu modeliyle birleştirir ve kullanıcının kendi cümlesiyle sorduğu soruya, sorunun kaynağı olan parçayı göstererek cevap verir.

Problem

Havacılık ve savunma sanayisinde teknik yayınlar **S1000D** standardıyla yazılır. Bilgi küçük XML modüllere bölünür; makineler için düzenli, insanlar için yorucudur. Bir arızanın kaynağını bulmak için doğru modülü bilmek, parça numaralarını çözmek ve çizimle metin arasında gidip gelmek gerekir.

Sıradan bir kullanıcı ise sorunu parça adıyla değil belirtiyle anlatır: "soğukken zor çalışıyor", "frende ses var". Bu iki dünya arasında köprü yoktur.

Amaç

1. **Görerek anlamak**
   Her parça 3D modelde yerinde görünür; işlevi, arızaları ve tamir adımı tek dokunuşla açılır.
2. **Belirtiden parçaya**
   Yapay zekâ belirtiyi okur, olası parçaları kalibre olasılıklarla sıralar, en olasını gösterir.
3. **Taşınabilir bilgi**
   Doküman, model ve eşleşmeler tek bir `.naya` dosyasında paylaşılır.

Kapsam

BAŞLIK	BU PROJEDE
Platform	Flutter ile tek kod tabanı: web ve Android
Hazır şablonlar	Araba, bisiklet, tank, savaş uçağı
İçe aktarma	Dışarıdan gelen S1000D dosyalarının `.zip` olarak yüklenip taranması
Yapay zekâ	Sohbet için DeepSeek Flash, olasılık için Jev; ikisi de OpenRouter üzerinden, kullanıcının kendi anahtarıyla
Kapsam dışı	Gerçek askeri teknik veri; şablonlar açık kaynak modeller ve genel bakım bilgisiyle hazırlanır

Yöntem

01 · ARAŞTIRMA	02 · TASARIM	03 · UYGULAMA	04 · DOĞRULAMA
S1000D, açık kaynak 3D modeller ve olasılık modelleri incelendi.	Format, mimari, veritabanı ve arayüz kararları netleştirildi.	Altı adımlı takvimle, iki kişi sırayla geliştirir.	Birim testleri, Android cihaz testleri ve kullanıcı senaryoları.

02 Genel Tanıtım

NAYA iki yönlü çalışır. Bir tarafta bizim hazırladığımız, modeli ve eşleşmeleri tam şablonlar; diğer tarafta kullanıcının dışarıdan getirdiği S1000D içerikleri vardır. İkisi de aynı kütüphanede, aynı arayüzle açılır.

YÖN 1

Hazır şablonlar

S1000D dokümanları, parçalara ayrılmış gerçekçi bir 3D model ve her parçanın görevini, bölgesini, arızalarını anlatan eşleşme dosyaları birlikte gelir.

Araba · Bisiklet · Tank · Savaş uçağı

YÖN 2

Dışarıdan içe aktarma

Dağınık S1000D dosyaları `.zip` olarak yüklenir. Uygulama arşivi açar, modülleri tanır, parça ağacını çıkarır. Modeli olmayan içerik için sistem ağacından şematik bir 3D görünüm üretilir.

`.zip` · tarama · indeks · `.naya`



ŞEKİL 2.1 · BİR KULLANICININ YOLU: BELİRTİDEN PARÇAYA

A — Elle inceleme
Model döndürülür, parçalarına ayrılır, tek parça yalıtılıp incelenir.

B — Yapay zekâ sohbeti
Sorun anlatılır ya da bulunan parça hakkında soru sorulur.

C — Paylaşım
Her içerik `.naya` olarak indirilir ve başka cihazda açılır.

Kimler için

ÖĞRENCİ

Bir sistemin parçalarını ve birbirine nasıl bağlandığını görerek öğrenir.

TEKNİSYEN

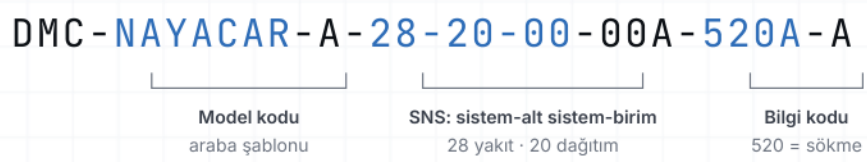
Arızanın olası kaynağına ve ilgili prosedüre hızlıca ulaşır.

ARAÇ SAHİBİ

Ustaya gitmeden önce sorunun ne olabileceğini anlar.

03 S1000D ve .naya Formatı

S1000D, teknik yayınları tekrar kullanılabilir küçük parçalara bölen uluslararası bir standarttır. NAYA bu parçalara dokunmaz; üzerine 3D modeli ve parça eşleşmelerini ayrı bir katman olarak ekler.



ŞEKİL 3.1 · BİR DATA MODULE KODUNUN OKUNUŞU (ÖRNEK)

Kullandığımız kavramlar

TERİM	ANLAMI
CSDB	Tüm modüllerin tutulduğu ortak kaynak veritabanı
DM	Data module: tek konuyu anlatan XML parçası
DMC	Her modülün benzersiz kodu (Şekil 3.1)
SNS	Sistem numaralandırma: sistem, alt sistem, birim
IPD	Resimli parça listesi; çizimdeki numaralı parçalar
ICN	Çizim, fotoğraf ve diğer medya dosyaları

.naya dosyasının içi

```
araç.naya          ZIP kabı
├── mimetype         ilk kayıt
├── naya.json        manifest
├── csdb/            ham S1000D
├── model/
│   ├── product.glb  ana model
│   └── detail/      gerektikçe
├── map/             NAYA dili
│   ├── parts.json
│   ├── faults.json
│   ├── zones.json
│   └── hotspots.json
├── media/
└── ai/chunks.jsonl  isteğe bağlı
```

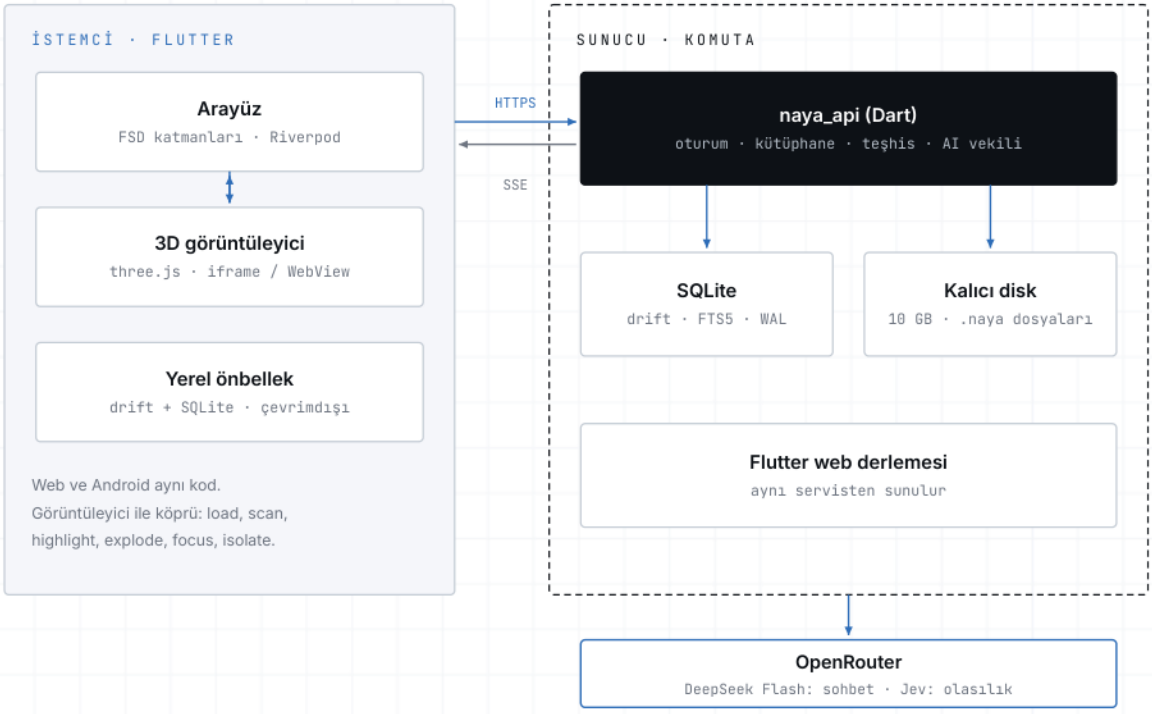
1. **S1000D değişmez**
   Dosya her zaman tekrar saf S1000D olarak dışa aktarılabilir.

2. **Tek parça ağacı**
   `parts.json` içindeki üst parça bağı hem animasyonu hem teşhisi besler.

3. **Hafif model**
   glTF, meshopt ve webp ile şablon başına 50 MB altı.

04 Sistem Mimarisi

Sistem üç katmandan oluşur: Flutter ile yazılmış istemci, Dart ile yazılmış küçük bir sunucu ve dış yapay zekâ servisi. İstemci internet yokken de açık dosyaları gösterebilir; sohbet ve senkronizasyon sunucu üzerinden çalışır.



ŞEKİL 4.1 · BİLEŞENLER VE VERİ AKIŞI

1. **Anahtar sunucuda**
   Kullanıcının OpenRouter anahtarı AES-GCM ile şifrelenip saklanır, tarayıcıya geri dönmez.

2. **Yerelde aynı yapı**
   Docker yalnız yayın için; geliştirirken `dart run` ve `flutter run` yeterli.

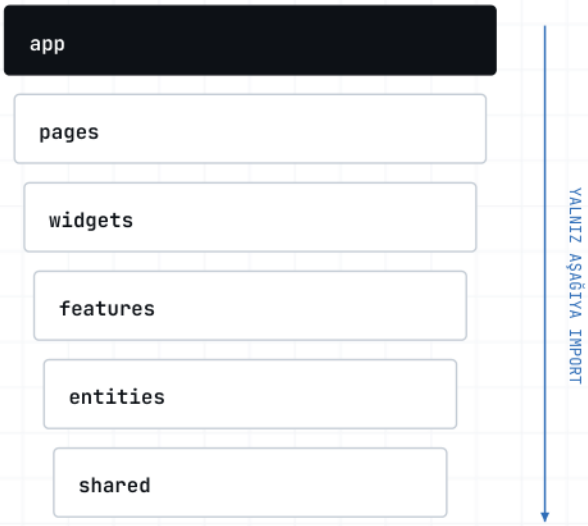
3. **Düşük maliyet**
   Tek servis ve 10 GB disk: aylık yaklaşık 11 dolar.

Flutter ile görüntüleyici arasındaki köprü

MESAJ	YÖN	NE YAPAR
`load`	Flutter → 3D	.naya içindeki modeli ve parça ağacını yükler
`scan · highlight`	Flutter → 3D	Parçaları sırayla tarar, olasılığa göre parlatır
`explode · focus`	Flutter → 3D	Modeli ayırır, kamerayı seçilen parçaya götürür
`isolate · xray`	Flutter → 3D	Tek parçayı yalıtır ya da gövdeyi saydam yapar
`onPartTap`	3D → Flutter	Dokunulan parçanın kimliğini gönderir, parça kartı açılır

05 Flutter Katmanı

Uygulama tek Flutter kod tabanıdır. Klasörler Feature-Sliced Design yaklaşımının Flutter'a uyarlanmış hâliyle düzenlenir; her katman yalnız kendisinden aşağıdakileri kullanabilir ve bu kural lint ile denetlenir.



app	Başlangıç, yönlendirme, tema, bağımlılıklar
pages	Ekranlar: kütüphane, model sayfası, ayarlar
widgets	Büyük bloklar: görüntüleyici, sohbet, parça kartı
features	Kullanıcı eylemleri: teşhis et, içe aktar, sor
entities	İş nesneleri: ürün, parça, arıza, data module
shared	Ortak araçlar: arayüz kiti, API istemcisi

ŞEKİL 5.1 · KATMANLAR

Temel paketler

PAKET	GÖREV
flutter_riverpod	Durum yönetimi
go_router	Sayfa yönlendirme, derin bağlantı
drift	SQLite şeması; web'de WASM ile
webview_flutter	Android'de 3D görüntüleyici
HtmlElementView	Web'de görüntüleyici iframe'i
flutter_secure_storage	Android'de oturum token'ı
archive · xml	ZIP ve S1000D okuma
video_player	Açılış animasyonu

Platform farkları

Web: görüntüleyici iframe içinde çalışır, oturum httpOnly çerezde durur, önbellek tarayıcının OPFS alanındadır.

Android: görüntüleyici WebView içinde çalışır, token güvenli depolamada durur, dosyalar uygulama klasöründedir.

İki platformda da 3D sahne aynı three.js kodudur; Flutter tarafı yalnızca küçük bir mesaj köprüsü konuşur.

06 Veritabanı

Sunucu tarafında tek bir SQLite dosyası kullanılır. Okul projesi ölçeğinde hızlı, kurulum gerektirmeyen ve yedeklemesi kolay bir seçimdir. Şema drift ile Dart kodunda tanımlanır; aynı şema uygulamanın yerel önbelleğinde de kullanılır.



ŞEKİL 6.1 · TABLOLAR VE İLİŞKİLER

1 WAL modu
Okuma ve yazma birbirini kilitlemez.

2 FTS5 arama
Dokümanlarda tam metin arama, ek servis yok.

3 Günlük yedek
`VACUUM INTO` ile son 7 kopya.

4 Sırlar korunur
Şifre Argon2id, token SHA-256, anahtar AES-GCM.

ÖRNEK · DİAGNOSES.RESULT

```
{
  "system": { "yakit": 0.71, "ateslem": 0.18, "hicbiri": 0.11 },
  "part":   { "yakit.pompa": 0.62, "yakit.filtre": 0.24 },
  "fault":  "f.yakitpompa.asinma",
  "procedure": "DMC-NAYACAR-A-28-20-00-00A-520A-A",
  "safety": 0.34,
  "model": "jev-1.13"
}
```

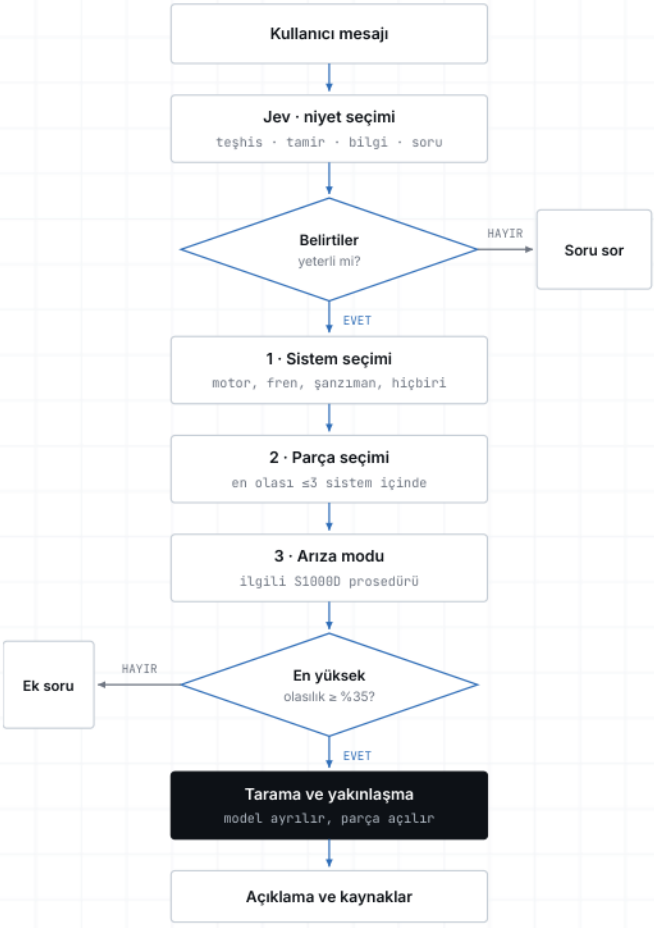
Neden SQLite

Tek dosya, ayrı bir veritabanı sunucusu yok. Yerelde kurulum gerektirmez; yayında kalıcı diskte durur. Proje büyürse aynı drift şemasıyla PostgreSQL'e geçilebilir.

Örnekteki olasılıklar gösterim amaçlıdır.

07 Çalışma Mantığı ve Algoritmalar

Teşhiste olasılığı sohbet modeline hesaplatmak her seferinde farklı sonuç verir. Bu yüzden iki model ayrı işler yapar: Jev belirtileri kalibre olasılıklara çevirir, DeepSeek Flash sonucu insan diliyle anlatır.



ŞEKİL 7.1 · TEŞHİS AKIŞI

Olasılık birleştirme

`P(parça) = P(sistem) × P(parça | sistem)`

Önce sistem, sonra o sistemin parçaları seçilir. İki aşamalı seçim, Jev'in tek soruda en fazla 255 seçenek sınırına takılmadan yüzlerce parçayı sıralamasını sağlar. Her aşamada "hiçbiri" seçeneği bulunur; model emin değilse bunu söyleyebilir.

İçe aktarma algoritması

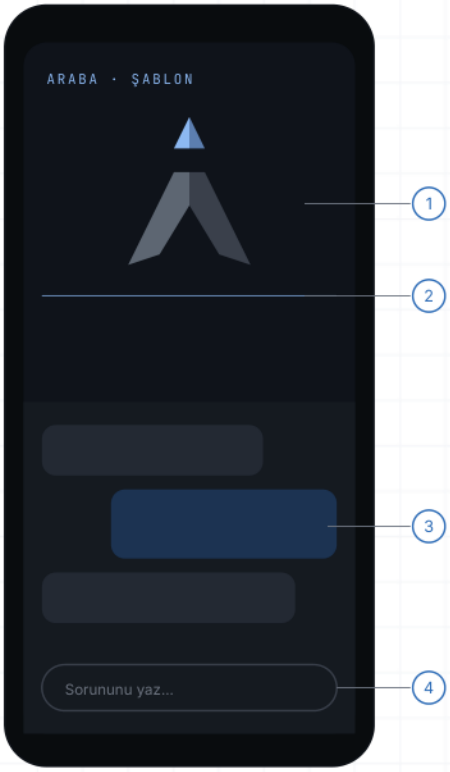
1. **Aç** · ZIP okunur, dosyalar türlerine ayrılır.
2. **Tanı** · Her DM'in kodu çözülür, issue sürümü okunur.
3. **Ağaç** · SNS'ten sistem ağacı, IPD'den parça listesi çıkar.
4. **İndeks** · Metin bölünür, FTS5'e yazılır.
5. **Paketle** · Şematik model ile `.naya` oluşur.

Neden Jev

Jev metin üretmez; seçeneklere olasılık döndürür. %90 dediği şeyin gerçekte yaklaşık %90 doğru çıkması hedeflenerek eğitilmiştir. Bir teşhis yaklaşık 15 bin token, maliyeti 1 sentin altında.

08 Arayüz ve Kullanıcı Akışı

Arayüz koyu ve açık temayı destekler; vurgu rengi çelik mavisidir. Ana ekran olan model sayfasında üstte 3D model, altta yapay zekâ sohbeti yer alır. Kullanıcı modeli elle inceleyebilir; bir soru sorduğunda tarama başlar ve model bulunan parçaya yaklaşır.



ŞEKİL 8.1 · MODEL SAYFASI

1 3D model, elle döndürülür ve ayrılır
2 Tarama ışığı
3 Yapay zekâ sohbeti
4 Mesaj alanı

Ekranlar

EKRAN	İÇERİK
Açılış	Logo animasyonu, sonra kütüphane
Kütüphane	Şablonlar ve kullanıcının dosyaları
Model sayfası	Üstte 3D model, altta sohbet
Parça kartı	İşlev, arızalar, tamir adımları, görseller
DM okuyucu	S1000D adımlarını okunur gösterir
İçe aktarma	ZIP yükleme sihirbazı
Teşhis geçmişi	Önceki teşhisler ve olasılıklar
Ayarlar	AI anahtarı, model seçimi, tema

Model üzerindeki etkileşimler

HAREKET	SONUÇ
Sürükle	Model döner
İki parmak	Yakınlaşır, uzaklaşır
Parçaya dokun	Parça kartı açılır
Parçala düğmesi	Model montaj ağacına göre ayrılır
Uzun bas	Parça yalıtılır, gövde saydamlaşır

09 Dosya Yapısı

Tüm kod tek bir depoda durur. Dart paketleri ortak bir çalışma alanında toplanır; .naya okuyucu ve S1000D ayrıştırıcı saf Dart olduğu için hem uygulama, hem sunucu, hem de komut satırı aracı aynı kodu kullanır.

DEPO

```
naya/
├── apps/
│   ├── naya/              Flutter: web + Android
│   └── naya_api/          Dart sunucu
├── packages/
│   ├── naya_format/       .naya oku, yaz, doğrula
│   ├── naya_s1000d/       S1000D ayrıştırıcı
│   ├── naya_diagnosis/    teşhis akışı
│   └── viewer_web/        three.js görüntüleyici
├── tools/naya_cli/        pack · validate · import
├── templates/src/         şablon kaynakları
├── schemas/               JSON Schema
├── assets/brand/          logo, açılış, yükleme
├── PLAN.md
└── README.md
```

FLUTTER UYGULAMASI

```
apps/naya/lib/
├── app/          router, tema, DI
├── pages/
│   ├── library/
│   ├── workspace/
│   ├── import/
│   └── settings/
├── widgets/
│   ├── viewer_panel/
│   ├── chat_panel/
│   └── part_sheet/
├── features/
│   ├── diagnose/
│   ├── import_zip/
│   └── ask_ai/
├── entities/     product, part, fault
└── shared/       ui, api, utils
```

1 **İsimler İngilizce**
Sınıf ve fonksiyon adları İngilizce, yorumlar ve belgeler Türkçe.

2 **Katman kuralı**
Bir katman yalnız kendinden aşağıdakileri import eder; lint denetler.

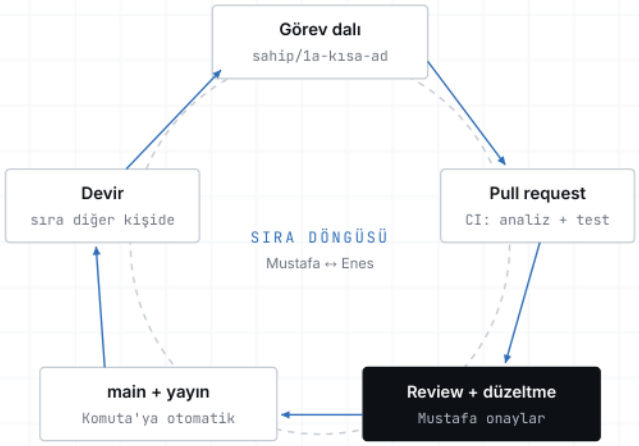
3 **Test yanında**
Format, ayrıştırıcı ve teşhis paketleri birim testleriyle gelir.

Kim neyi kullanır

PAKET	APPS/NAYA	APPS/NAYA_API	TOOLS/NAYA_CLI
naya_format	✓	✓	✓
naya_s1000d	✓	✓	✓
naya_diagnosis	✓	✓	·
viewer_web	✓	·	·

10 Çalışma Şekli

Proje çevik bir modelle, iki kişi sırayla çalışarak ilerler. Bir adımın görevleri iki kişiye bölünür; biri kendi kısmını bitirip devrettiğinde sıra diğerine geçer. Çalışma kanalımız Notion'dur.



ŞEKİL 10.1 · BİR GÖREVİN YOLU

GELİŞTİRME MODELİ

Çevik · Kanban yaklaşımlı artımlı model

Kanban: görevler bekliyor, yapılıyor, incelemede, bitti sırasıyla akar; aynı anda tek görev açıktır.

Artımlı: uygulama altı adımda büyür; her adım çalışan bir parça bırakır.

XP uygulamaları: küçük PR'lar, kod incelemesi, her PR'da otomatik test.

Mimari ve dosya formatı kodlamadan önce kesinleştirildiği için model, önden tasarımlı bir hibrittir.

ÇALIŞMA KANALI

Notion

Görevler, sıra ve proje notları Notion üzerinden takip edilir.

Değişmez kurallar

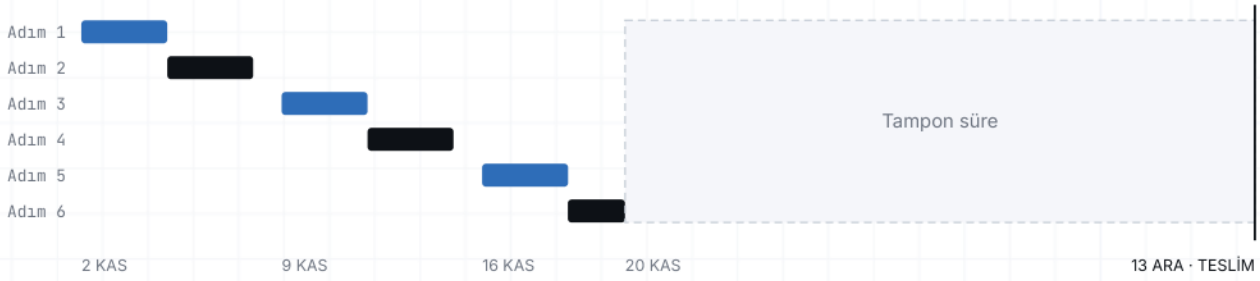
1. **Her görev kendi dalında**
   Dal adı sahip, görev numarası ve kısa addan oluşur; her görev için ayrı PR açılır.
2. **Review olmadan main yok**
   Hiçbir PR, review ve istenen düzeltmeler bitmeden ana dala alınmaz.
3. **Otomatik kontrol**
   Her PR'da analiz ve testler çalışır; ana dala giren kod web ve APK olarak derlenir.
4. **Her iş belgelenir**
   Biten iş kısaca belgelenir ve sıra diğer kişiye devredilir.

11 Görev Dağılımı ve Takvim

Çalışma 2 Kasım 2026 Pazartesi başlar ve 20 Kasım'da bitecek şekilde altı adıma bölünmüştür. Son teslim 13 Aralık 2026; aradaki süre tampondur. Model, ilk tasarım ve kurulum işleri Mustafa'da; tasarımın devamı, Android ve test işleri Enes'tedir.

	ADIM	MUHAMMET MUSTAFA ZAHTER	ENES YENİÇIRAK
1	**Kurulum ve format** 2 - 4 Kasım	1A Monorepo iskeleti, lint 1B .naya formatı v1 1C İlk tasarım taslağı	1D Flutter ve Android ortamı, CI 1E Format testleri
2	**S1000D ve görüntüleyici** 5 - 7 Kasım	2A S1000D ayrıştırıcı 2B three.js görüntüleyici ve köprü	2C Uygulama iskeleti, kütüphane 2D Ayrıştırıcı testleri
3	**Sunucu ve ilk şablon** 9 - 11 Kasım	3A Bisiklet şablonu 3B Görüntüleyici efektleri	3C naya_api, oturum, dosyalar 3D Model sayfası, parça kartı
4	**Yapay zekâ ve teşhis** 12 - 14 Kasım	4A AI hattı, Jev teşhisi 4B Teşhis ve tarama bağlantısı	4C Ayarlar, sohbet, geçmiş 4D Komuta yayını, Android testi
5	**Şablonlar ve içe aktarma** 16 - 18 Kasım	5A Araba şablonu 5B Tank ve uçak şablonları	5C ZIP içe aktarma, dışa aktarma 5D Uçtan uca test turu
6	**Cila ve teslim** 19 - 20 Kasım	6A Tasarım kalite turu 6C Rapor ve sunum (birlikte)	6B Performans, erişilebilirlik 6C Rapor ve sunum (birlikte)

Zaman çizelgesi



MUHAMMET MUSTAFA ZAHTER

3D modeller ve şablonlar, ilk tasarım taslağı ve marka, .naya formatı ve S1000D ayrıştırıcı, ilk kurulum, görüntüleyici, yapay zekâ hattı, pull request review.

ENES YENİÇIRAK

Tasarımın uygulamada devamı, Flutter ekranları, Android ortamı ve testleri, sunucu ve oturum, içe aktarma, başlangıçta test sorumluluğu.

12 Sonuç ve Kaynaklar

NAYA, standart bir teknik yayının içindeki bilgiyi aracın kendisi üzerinde gösterilebilir hâle getirir. Kullanıcı parça numarası bilmeden derdini anlatır; sistem olası parçayı ölçülebilir bir güvenle gösterir ve kaynak prosedürü yanına koyar.

Beklenen çıktılar

1. Web ve Android'de çalışan NAYA uygulaması
2. Dört hazır şablon ve `.naya` formatı v1
3. Yayında bir sunucu ve web adresi
4. Adım adım geliştirme dokümanları

Riskler ve önlemler

RİSK	ÖNLEM
İç parça modeli bulunamaması	Dış gövde ve alt montajlar ayrı açık kaynak modellerden birleştirilir
Android WebView performansı	Sıkıştırılmış model ve gerektiğinde yüklenen detaylar
Yapay zekâ hatası	Prosedürler yalnız dokümandan; düşük güvende ek soru

BÜYÜME SENARYOSU

NAYA ileride bir paylaşım platformuna dönüşebilir: herkes kendi `.naya` dosyasını tasarlayıp yükler, S1000D paketlerini paylaşır; diğer kullanıcılar bunları keşfeder, kullanır ve geliştirir.

Kaynaklar

[1] **S1000D** · International specification for technical publications, Issue 5.0. ASD/AIA/ATA.

[2] **Flutter** · Flutter documentation, flutter.dev

[3] **three.js** · three.js documentation ve glTF 2.0 belirtimi, Khronos Group

[4] **Jev** · TypeSafe AI, Jev System One API belgeleri

[5] **OpenRouter** · OpenRouter API ve model listesi belgeleri

[6] **SQLite FTS5** · SQLite Full-Text Search Extension, sqlite.org

[7] **drift** · drift: reactive persistence library for Dart

[8] **FSD** · Feature-Sliced Design metodolojisi, feature-sliced.design

NAYA